

Guía Definitiva de Programación

Paso a paso para agregar funciones y módulos al Backend y al Frontend

Sistema de novedades de empleados · MAQUINOR · 2026

Contenido

1.	Arquitectura del sistema	3
2.	Stack tecnológico	4
3.	Estructura de carpetas	5
4.	Agregar un módulo al BACKEND	6
4.1	Crear la tabla (migración)	6
4.2	Crear el repositorio	7
4.3	Crear el handler (mutaciones)	8
4.4	Registrar el comando	9
4.5	Crear la ruta REST	9
4.6	Registrar la ruta en index.js	10
5.	Agregar un módulo al FRONTEND	11
5.1	Crear las funciones de API	11
5.2	Crear el componente de página	12
5.3	Registrar la ruta en App.jsx	15
5.4	Agregar al menú lateral (Layout)	15
6.	Ejemplo completo: módulo Vacaciones	16
7.	Patrones de uso frecuente	20
8.	Comandos útiles de desarrollo	22
9.	Reglas de oro	23

1. Arquitectura del sistema

novedad-app sigue una arquitectura **cliente-servidor** desacoplada. El frontend React se comunica con el backend Express mediante HTTP. Las **mutaciones** (crear, modificar) pasan por un **Command Bus**; las **consultas** (listar, obtener) van directo a rutas REST.

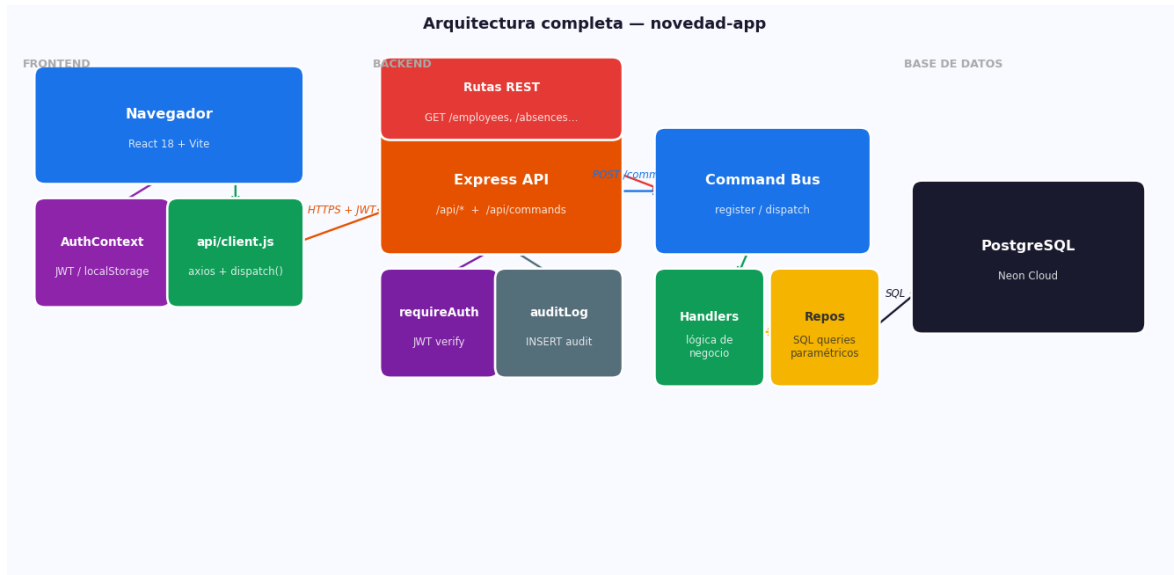
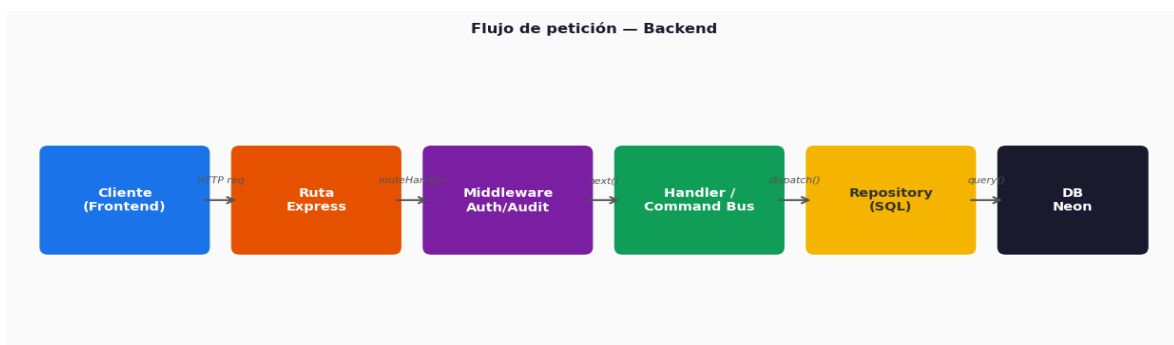


Diagrama de arquitectura completa — novedad-app

Dos patrones de comunicación

■ Regla clave: usa el Command Bus para operaciones que **modifican estado** (crear, actualizar, dar de baja). Usa rutas REST directas para **consultas** (GET).

Patrón	Cuándo usarlo	Endpoint	Ejemplo
Command Bus	Crear / Modificar / Borrar	POST /api/commands	RegisterAbsence, OnboardEmployee
REST directo	Consultar / Listar / Exportar	GET /api/recurso	GET /api/absences, GET /api/employees



Flujo de una petición en el backend

Flujo de acción — Frontend



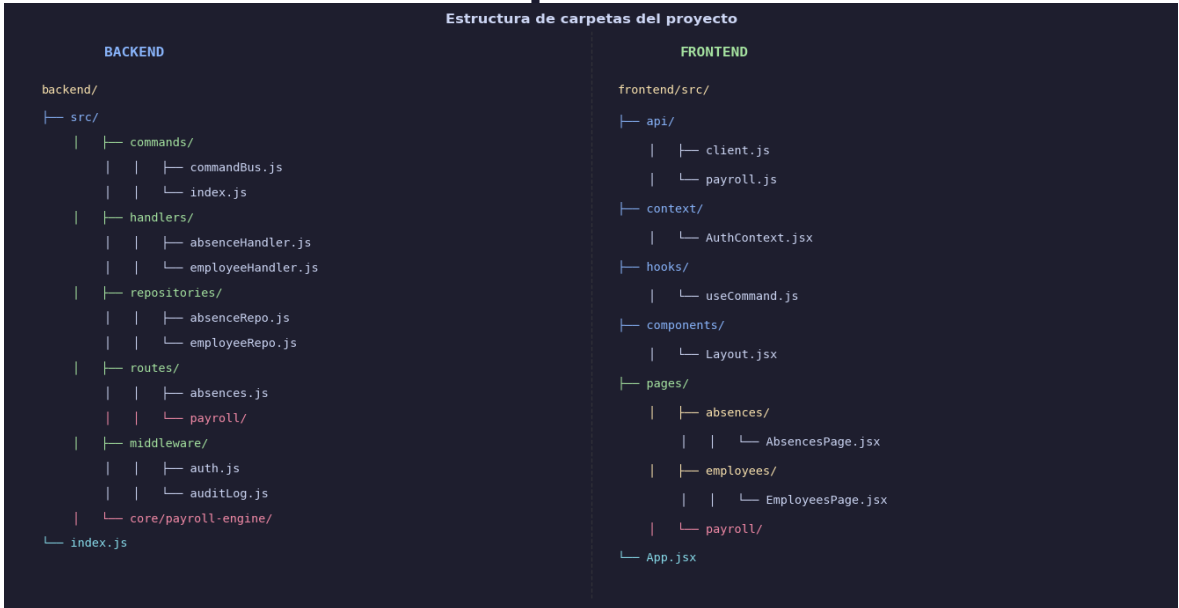
Flujo de una acción en el frontend

2. Stack tecnológico

Capa	Tecnología	Versión	Deploy
Frontend	React + Vite + Tailwind CSS	React 18 / Vite 5 / TW 4	Vercel
Routing	React Router	v6	—
HTTP cliente	axios	—	—
Backend	Node.js + Express	Node 20 / Express 4	Render
Base de datos	PostgreSQL (sin ORM)	—	Neon Cloud
Auth	JWT + bcryptjs	—	—
Build tool	Vite (frontend)	—	—

■ Los repos de frontend y backend están **separados**: cada uno tiene su propio **.git** y se despliega independientemente.

3. Estructura de carpetas



Árbol de carpetas — backend (izq.) y frontend (der.)

Archivos clave que debes conocer

Archivo	Rol
backend/src/commands/commandBus.js	Registro y despacho de comandos
backend/src/commands/index.js	Registra todos los comandos del sistema
backend/src/middleware/auth.js	requireAuth + requireRole
backend/src/db/client.js	Pool de conexiones a PostgreSQL (Neon)
backend/index.js	App Express: monta todas las rutas
frontend/src/api/client.js	axios + dispatch() para commands
frontend/src/api/payroll.js	Todas las llamadas REST de nómina
frontend/src/hooks/useCommand.js	Hook para ejecutar comandos con estado
frontend/src/context/AuthContext.jsx	Login, logout, token JWT
frontend/src/App.jsx	Rutas y componentes del SPA
frontend/src/components/Layout.jsx	Shell con menú lateral

4. Agregar un módulo al BACKEND

Ejemplo: módulo **Permisos de trabajo** (*work_permits*). Sigue estos 6 pasos en orden.

Paso 4.1 — Crear la tabla — Migración SQL

Crea un archivo en **backend/** llamado `migrate_work_permits.js`:

```
// migrate_work_permits.js
// backend/migrate_work_permits.js
require('dotenv').config()
const { query } = require('../src/db/client')

async function migrate() {
  await query(`
  CREATE TABLE IF NOT EXISTS work_permits (
    id SERIAL PRIMARY KEY,
    employee_id INTEGER NOT NULL REFERENCES employees(id),
    type VARCHAR(50) NOT NULL,
    start_date DATE NOT NULL,
    end_date DATE,
    reason TEXT,
    created_by INTEGER REFERENCES users(id),
    created_at TIMESTAMPTZ DEFAULT NOW()
  )
  `)
  console.log('Tabla work_permits creada')
  process.exit(0)
}

migrate().catch(e => { console.error(e); process.exit(1) })
```

Agrega el script en **package.json** del backend:

```
// backend/package.json - scripts
"migrate:work-permits": "node migrate_work_permits.js"
```

Ejecuta la migración:

```
// Terminal
cd backend
npm run migrate:work-permits
```

Paso 4.2 — Crear el repositorio

Crea **backend/src/repositories/workPermitRepo.js**. Sólo SQL puro — sin lógica de negocio:

```
// workPermitRepo.js
// backend/src/repositories/workPermitRepo.js
const { query } = require('../db/client')
```

```

const workPermitRepo = {
  async create(d) {
    const { rows } = await query(
      `INSERT INTO work_permits
      (employee_id, type, start_date, end_date, reason, created_by)
      VALUES ($1,$2,$3,$4,$5,$6) RETURNING *`,
      [d.employeeId, d.type, d.startDate, d.endDate, d.reason, d.createdBy]
    )
    return rows[0]
  },
  async findAll() {
    const { rows } = await query(`
    SELECT wp.*, e.name AS employee_name
    FROM work_permits wp
    JOIN employees e ON e.id = wp.employee_id
    ORDER BY wp.created_at DESC
    `)
    return rows
  },
  async findById(id) {
    const { rows } = await query(
      `SELECT * FROM work_permits WHERE id = $1`, [id]
    )
    return rows[0] || null
  },
}

module.exports = workPermitRepo

```

Paso 4.3 — Crear el handler (lógica de negocio)

El handler valida los datos de entrada y llama al repositorio. Crea **backend/src/handlers/workPermitHandler.js**:

```

// workPermitHandler.js
// backend/src/handlers/workPermitHandler.js
const workPermitRepo = require('../../repositories/workPermitRepo')

async function registerWorkPermit(payload, context) {
  const { employeeId, type, startDate, endDate, reason } = payload

  // Validación de campos requeridos
  if (!employeeId || !type || !startDate) {
    const e = new Error('employeeId, type y startDate son requeridos')
    e.status = 400
    throw e
  }

  return workPermitRepo.create({

```



```

    employeeId, type, startDate, endDate, reason,
    createdBy: context.userId,
  })
}

module.exports = { registerWorkPermit }

```

Paso 4.4 — Registrar el comando en commands/index.js

Abre **backend/src/commands/index.js** y agrega dos líneas:

```

// commands/index.js
// backend/src/commands/index.js
const { register } = require('./commandBus')
const { registerAbsence } = require('../handlers/absenceHandler')
const { registerAccident } = require('../handlers/accidentHandler')
const { changeShift } = require('../handlers/shiftHandler')
const { onboardEmployee } = require('../handlers/employeeHandler')

// ↓ AGREGAR ESTAS DOS LÍNEAS
const { registerWorkPermit } = require('../handlers/workPermitHandler')

register('RegisterAbsence', registerAbsence)
register('RegisterAccident', registerAccident)
register('ChangeShift', changeShift)
register('OnboardEmployee', onboardEmployee)

// ↓ AGREGAR ESTA LÍNEA
register('RegisterWorkPermit', registerWorkPermit)

```

Paso 4.5 — Crear la ruta REST

Crea **backend/src/routes/workPermits.js** para las consultas (GET):

```

// routes/workPermits.js
// backend/src/routes/workPermits.js
const router = require('express').Router()
const repo = require('../repositories/workPermitRepo')
const { requireAuth } = require('../middleware/auth')
const { auditRead } = require('../middleware/auditLog')

// GET /api/work-permits – lista todos
router.get('/', requireAuth, auditRead('WorkPermits'), async (req, res, next) => {
  try {
    res.json({ data: await repo.findAll() })
  } catch (err) { next(err) }
})

// GET /api/work-permits/:id – obtiene uno
router.get('/:id', requireAuth, async (req, res, next) => {
  try {

```

```
const permit = await repo.findById(req.params.id)
if (!permit) return res.status(404).json({ error: 'No encontrado' })
res.json({ data: permit })
} catch (err) { next(err) }
})

module.exports = router
```

Paso 4.6 — Registrar la ruta en backend/index.js

Abre **backend/index.js** y agrega el require + app.use:

```
// backend/index.js
// backend/index.js (fragmento relevante)
const absencesRouter = require('./src/routes/absences')
const accidentsRouter = require('./src/routes/accidents')
const employeesRouter = require('./src/routes/employees')

// ↓ AGREGAR
const workPermitsRouter = require('./src/routes/workPermits')
// ...

app.use('/api/absences', requireAuth, absencesRouter)
app.use('/api/accidents', requireAuth, accidentsRouter)
app.use('/api/employees', requireAuth, employeesRouter)

// ↓ AGREGAR
app.use('/api/work-permits', requireAuth, workPermitsRouter)
```

■ El backend ya está listo. Prueba con: **POST /api/commands** body: `{"command": "RegisterWorkPermit", "payload": {...}}` y **GET /api/work-permits**.

5. Agregar un módulo al FRONTEND

Continuando con el ejemplo de **Permisos de trabajo**. Sigue estos 4 pasos para tener la página funcional.

Paso 5.1 — Crear las funciones de API

Si el módulo no es de nómina, créalo como archivo nuevo. Si es de nómina, agrégalo a **frontend/src/api/payroll.js**. Para este ejemplo crea **frontend/src/api/workPermits.js**:

```
// api/workPermits.js

// frontend/src/api/workPermits.js

import http, { dispatch } from './client'

export const workPermits = {

  // Consultas → GET directo

  list: () =>

    http.get('/work-permits').then(r => r.data.data),

  get: (id) =>

    http.get(`/work-permits/${id}`).then(r => r.data.data),

  // Mutaciones → Command Bus

  create: (payload) =>

    dispatch('RegisterWorkPermit', payload),

}
```

Paso 5.2 — Crear el componente de página

Crea la carpeta y el archivo **frontend/src/pages/workPermits/WorkPermitsPage.jsx**. La estructura estándar de una página tiene 4 secciones:

```
// WorkPermitsPage.jsx
// frontend/src/pages/workPermits/WorkPermitsPage.jsx

import { useState, useEffect } from 'react'
import { workPermits } from '../../../api/workPermits'
import { useCommand } from '../../../hooks/useCommand'

// ■■ 1. ESTADO
export default function WorkPermitsPage() {
  const [permits, setPermits] = useState([])
  const [loading, setLoading] = useState(true)
  const [showModal, setModal] = useState(false)
  const [form, setForm] = useState({
    employeeId: '', type: '', startDate: '', endDate: '', reason: '',
  })

  const createCmd = useCommand('RegisterWorkPermit')

  // ■■ 2. CARGA INICIAL
  useEffect(() => { load() }, [])

  async function load() {
    setLoading(true)
  }
}
```

```
try { setPermits(await workPermits.list()) }  
finally { setLoading(false) }  
}  
  
// ■■ 3. ACCIONES  
async function handleSubmit(e) {  
  e.preventDefault()  
  await createCmd.execute(form)  
  setModal(false)  
  setForm({ employeeId:'', type:'', startDate:'', endDate:'', reason:'' })  
  load()  
}  
  
// ■■ 4. RENDER  
return (  
  <div className='p-6'>  
    <div className='flex justify-between items-center mb-4'>  
      <h1 className='text-2xl font-bold'>Permisos de Trabajo</h1>  
      <button onClick={() => setModal(true)}  
        className='bg-blue-600 text-white px-4 py-2 rounded'>  
+ Nuevo permiso  
      </button>  
    </div>  
    { /* Tabla */}  
    [loading ? <p>Cargando...</p> : (  
      <table className='w-full border-collapse text-sm'>  
        <thead>  
          <tr className='bg-gray-100'>  
            <th className='p-2 text-left'>Empleado</th>  
            <th className='p-2 text-left'>Tipo</th>  
            <th className='p-2 text-left'>Inicio</th>  
            <th className='p-2 text-left'>Fin</th>  
          </tr>  
        </thead>  
        <tbody>  
          {permits.map(p => (  
            <tr key={p.id} className='border-b'>  
              <td className='p-2'>{p.employee_name}</td>  
              <td className='p-2'>{p.type}</td>  
              <td className='p-2'>{p.start_date?.slice(0,10)}</td>  
              <td className='p-2'>{p.end_date?.slice(0,10) || '-'}</td>  
            </tr>  
          ))}  
        </tbody>  
      </table>  
    )}  
    { /* Modal */}
```

```

{showModal && (
  <div className='fixed inset-0 bg-black/40 flex items-center justify-center'>
    <form onSubmit={handleSubmit}
      className='bg-white rounded-xl p-6 w-[460px] space-y-4'>
      <h2 className='text-lg font-bold'>Nuevo permiso</h2>
      <input type='number' placeholder='ID Empleado'
        className='border rounded p-2 w-full'
        value={form.employeeId}
        onChange={e => setForm({...form, employeeId: e.target.value})} />
      <input type='text' placeholder='Tipo'
        className='border rounded p-2 w-full'
        value={form.type}
        onChange={e => setForm({...form, type: e.target.value})} />
      <input type='date'
        className='border rounded p-2 w-full'
        value={form.startDate}
        onChange={e => setForm({...form, startDate: e.target.value})} />
      <div className='flex gap-2 justify-end'>
        <button type='button' onClick={() => setModal(false)}
          className='px-4 py-2 border rounded'>Cancelar</button>
        <button type='submit'
          disabled={createCmd.loading}
          className='px-4 py-2 bg-blue-600 text-white rounded'>
          {createCmd.loading ? 'Guardando...' : 'Guardar'}
        </button>
      </div>
    </div>
    {createCmd.error && (
      <p className='text-red-500 text-sm'>{createCmd.error}</p>
    )}
  </form>
</div>
)}
</div>
)
}

```

Paso 5.3 — Registrar la ruta en App.jsx

Abre **frontend/src/App.jsx** y agrega el import y la ruta:

```

// App.jsx
// frontend/src/App.jsx — solo las líneas nuevas

// ↓ AGREGAR el import
import WorkPermitsPage from '../pages/workPermits/WorkPermitsPage'

// ↓ AGREGAR dentro de <Routes>
<Route

```

```
path='/work-permits'  
element={<PrivateRoute><WorkPermitsPage /></PrivateRoute>}  
/>
```

Paso 5.4 — Agregar al menú lateral (Layout.jsx)

Abre **frontend/src/components/Layout.jsx** y agrega el enlace al menú:

```
// Layout.jsx  
// frontend/src/components/Layout.jsx – fragmento del menú  
import { Link } from 'react-router-dom'  
  
// Dentro del array de items de navegación, agrega:  
{ path: '/work-permits', label: 'Permisos', icon: '■' },  
  
// O si el menú es JSX directo, agrega un <Link> en la lista:  
<Link to='/work-permits'  
  className='flex items-center gap-2 px-3 py-2 rounded hover:bg-gray-100'  
<span>■</span>  
{!collapsed && <span>Permisos</span>}  
</Link>
```

■ Con estos 4 pasos el módulo está completo. El frontend puede crear permisos (vía Command Bus) y listarlos (vía GET REST).

6. Ejemplo completo — módulo Vacaciones

A continuación se muestra el checklist completo de todos los archivos que debes crear o modificar para un módulo nuevo, usando **Vacaciones** como ejemplo real. Marca cada ítem al completarlo.

■	CREAR	backend/migrate_vacations.js	Tabla vacations en PostgreSQL
■	CREAR	backend/src/repositories/vacationRepo.js	create, findAll, findByEmployee
■	CREAR	backend/src/handlers/vacationHandler.js	registerVacation(payload, ctx)
■	EDITAR	backend/src/commands/index.js	require + register('RegisterVacation', ...)
■	CREAR	backend/src/routes/vacations.js	GET / y GET /:id
■	EDITAR	backend/index.js	app.use('/api/vacations', ...)
■	CREAR	frontend/src/api/vacations.js	list(), get(), create()
■	CREAR	frontend/src/pages/vacations/VacationPage.jsx	Página con tabla + modal
■	EDITAR	frontend/src/App.jsx	import + <Route path='/vacations' .../>
■	EDITAR	frontend/src/components/Layout.jsx	Link al menú
■	EJECUTAR	npm run migrate:vacations	Crear la tabla en Neon
■	PROBAR	POST /api/commands + GET /api/vacations	Verificar en Postman o el browser

Handler completo — vacationHandler.js

```
// vacationHandler.js
const vacationRepo = require('../repositories/vacationRepo')

async function registerVacation(payload, context) {
  const { employeeId, startDate, endDate, days } = payload
  if (!employeeId || !startDate || !endDate) {
    const e = new Error('employeeId, startDate y endDate requeridos')
    e.status = 400; throw e
  }
  return vacationRepo.create({
    employeeId, startDate, endDate, days,
    createdBy: context.userId,
  })
}

module.exports = { registerVacation }
```

Repositorio — vacationRepo.js

```
// vacationRepo.js
const { query } = require('../db/client')

const vacationRepo = {
  async create(d) {
```

```
const { rows } = await query(
  `INSERT INTO vacations (employee_id,start_date,end_date,days,created_by)
VALUES ($1,$2,$3,$4,$5) RETURNING *`,
  [d.employeeId, d.startDate, d.endDate, d.days, d.createdBy]
)
return rows[0]
},
async findAll() {
const { rows } = await query(`
SELECT v.*, e.name AS employee_name
FROM vacations v JOIN employees e ON e.id = v.employee_id
ORDER BY v.start_date DESC`,
)
return rows
},
async findByEmployee(employeeId) {
const { rows } = await query(
'SELECT * FROM vacations WHERE employee_id=$1 ORDER BY start_date DESC',
[employeeId]
)
return rows
},
}
module.exports = vacationRepo
```


7. Patrones de uso frecuente

7.1 useCommand — ejecutar una mutación con feedback

```
// Patrón useCommand
// En cualquier página del frontend:
import { useCommand } from '../hooks/useCommand'

const cmd = useCommand('NombreDelComando')

// Al enviar el formulario:
await cmd.execute({ campo1: valor1, campo2: valor2 })

// Propiedades disponibles:
cmd.loading // true mientras espera respuesta
cmd.error // string con el mensaje de error, o null
```

7.2 Llamadas GET directas desde un efecto

```
// Patrón GET con useEffect
import http from '../api/client'
import { useState, useEffect } from 'react'

const [data, setData] = useState([])

useEffect(() => {
  http.get('/employees')
    .then(r => setData(r.data.data))
    .catch(console.error)
}, [])
```

7.3 requireRole — proteger rutas del backend por rol

```
// Patrón requireRole
// backend/src/routes/miRuta.js
const { requireAuth, requireRole } = require('../middleware/auth')

// Solo accesible para admins:
router.delete('/:id',
  requireAuth,
  requireRole('admin'),
  async (req, res, next) => {
    // ...
  }
)
```

7.4 AdminRoute — proteger páginas del frontend por rol

```
// Patrón AdminRoute
// frontend/src/App.jsx
```

```
// Ya tienes AdminRoute definido. Úsalo así:
<Route
  path='/mi-pagina-admin'
  element={<AdminRoute><MiPaginaAdmin /></AdminRoute>}
/>
```

7.5 Errores estandarizados en handlers

```
// Patrón de errores
// Siempre lanza errores con .status para que Express los maneje:
if (!payload.campo) {
  const e = new Error('Descripción clara del error')
  e.status = 400 // 400 Bad Request, 404 Not Found, 403 Forbidden
  throw e
}
```

7.6 Agregar un parámetro configurable a payroll settings

```
// Patrón payroll_settings
-- En la DB, inserta el nuevo parámetro:
INSERT INTO payroll_settings (key, value, description)
VALUES ('mi_parametro', '0.05', 'Descripción del parámetro');

-- En el motor de nómina (backend/src/core/payroll-engine/pipeline/loadSettings.js):
// Ya se carga todo: ctx.settings.mi_parametro estará disponible

-- En la UI, ya aparece automáticamente en /payroll/settings
```

8. Comandos útiles de desarrollo

Contexto	Comando	Descripción
Backend	cd backend && npm run dev	Iniciar servidor (puerto 3001)
Backend	npm run migrate:nombre	Ejecutar una migración específica
Backend	npm run seed:settings	Actualizar payroll_settings
Frontend	cd frontend && npm run dev	Iniciar Vite (puerto 5173)
Frontend	npm run build	Build de producción
Git BE	git add . && git commit && git push	Commit y push del backend
Git FE	git add . && git commit && git push	Commit y push del frontend
DB	psql \$DATABASE_URL	Conexión directa a Neon
DB	\dt	Listar tablas (en psql)
Test API	curl -X POST localhost:3001/api/commands -H 'Content-Type: application/json' -d '{"command": "Register"	Protección de API

■ Nunca hagas **git push --force** a main. Cada subcarpeta (frontend/, backend/) tiene su propio repositorio.

9. Reglas de oro

Estas reglas mantienen el proyecto consistente y fácil de mantener:

- 01** **Command Bus solo para mutaciones**
Crear, actualizar y borrar van por POST `/api/commands` con `{command, payload}`. Las consultas (GET) van directo al router de Express.
- 02** **Sin ORM — SQL puro en los repos**
Los repositorios usan `query()` de `db/client.js` con parámetros posicionales (`$1`, `$2...`). Nunca concatenes strings con datos de usuario.
- 03** **Los parámetros de nómina van en `payroll_settings`**
Tasas, límites y valores configurables se leen de la tabla `payroll_settings`. Nunca los hardcodees en el código del motor.
- 04** **Validación en los handlers, no en las rutas**
El handler valida el payload y lanza errores con `.status`. La ruta solo llama `next(err)`.
- 05** **`useCommand` para mutaciones, `useEffect+http` para consultas**
En el frontend, sigue este patrón para que el estado de loading/error sea automático.
- 06** **Repos separados — commit a la subcarpeta correcta**
`frontend/` y `backend/` tienen cada uno su `.git`. No hagas git en la raíz del proyecto.
- 07** **Tailwind para estilos — sin CSS custom**
Usa clases utilitarias de Tailwind. Si necesitas algo muy específico usa `style={}`.
- 08** **Migra antes de codificar**
Crea y ejecuta la migración antes de escribir el repo o el handler. La tabla debe existir en Neon antes de cualquier query.